Branch Coverage Report

Decision-path execution and uncovered branch analysis

Event Management System (Eventify)

1. Objective

Branch coverage measures whether conditional decision outcomes were exercised by the automated tests. This report separates the overall backend branch result from the measured module-level gaps and the limited frontend module result.

2. Overall branch coverage

Codebase / scope	Branches covered	Total branches	Coverage
Backend (Jest coverage)	409	491	83.29%
Frontend registrationState.js	Not captured in absolute count	Not captured in absolute count	71.43%
Frontend all files combined	Not captured in absolute count	Not captured in absolute count	75.00% (includes test file)

The backend run left 82 of 491 measured branches uncovered. Frontend percentages are included only as reported by the experimental coverage run; absolute branch counts were not available in the retained output.

3. Backend module branch coverage

Module	Branch coverage	Interpretation / observed gap
Controllers aggregate	81.13%	Controller package overall is above 80%, but individual Admin controllers remain weak.
adminEventController	0% (0 / 16)	No measured conditional branches in this controller were covered by the automated tests.
adminUserController	0% (0 / 14)	No measured conditional branches in this controller were covered by the automated tests.
registrationIntegrity.js	50.00%	Transaction, connection-state, backfill and related conditional paths remain partially uncovered.
auditLogger.js	50.00%	Some conditional paths for actor/event details and logging behavior remain uncovered.
errorHandler.js	42.85%	Several error classification/response branches remain uncovered.

4. Risk assessment

- Admin controllers have the largest explicit branch gap: 30 of 30 measured branches were uncovered across the two listed controllers.
- Uncovered registration-integrity branches can conceal issues in transaction fallback, replica-set availability or recovery behavior; these paths should receive focused tests.
- Audit logger branches should be checked to ensure incomplete actor/event context or logger failures do not break the primary request path.
- Error-handler branches should be covered to verify consistent safe responses for known and unexpected errors.

5. Recommended branch-focused test additions

1. Admin event controller: test empty and populated event lists, valid/invalid identifiers, missing records, validation failures and unexpected repository errors.
2. Admin user controller: test list and lookup results, role update to each permitted role, invalid role, missing user, attempts to modify protected Admin roles and self-role restrictions.
3. Registration integrity: exercise available and unavailable replica-set states, transaction success/failure, duplicate/over-capacity branches and repair/backfill paths.

4. Audit logger: test complete and incomplete actor/event metadata and simulated logging failure.
5. Error handler: exercise each supported error class, malformed JSON and generic unexpected exceptions; assert status and response-body safety.

6. Conclusion

Overall backend branch coverage is 83.29%. The overall result is useful, but it masks concentrated gaps in the Admin controllers and several infrastructure/error-handling modules. Additional branch-focused tests are recommended before declaring full branch coverage of the planned critical paths.